מחלקת תקשוב · רשתות · מדריך למעביר השיעור

כלי תחקור ב-Windows

הסבר מלא לכל שקופית: מה כל פקודה בודקת, איך קוראים את הפלט שלה, מה התוצאה עדיין לא מוכיחה, ואיך מעבירים את סימולטור התחקור.

14	כ-55 דקות	6 שלבים
שקופיות	משך מומלץ	בסימולטור התחקור

לטכנאי שטח בסביבת Windows מנוהלת · כלים מובנים בלבד, בלי שינוי הגדרות

מטרה, הכנה וחלוקת זמן

למי המדריך הזה

המדריך כתוב למי שמעביר את השיעור, גם אם הוא לא עובד עם הכלים האלה כל יום. לכל שקופית יש הסבר רחב יותר ממה שמופיע על המסך: מה הפקודה עושה בפועל, אילו שדות בפלט חשובים, ואיפה טכנאים נוטים להסיק יותר ממה שהפלט מראה. לא צריך להקריא את הרקע. קחו ממנו את מה שמתאים לקבוצה, והשאירו את השאר לשאלות.

מטרת השיעור

שהטכנאי יבחר כלי לפי השאלה שעדיין פתוחה, ויקרא נכון את הפלט. בסוף השיעור חניך אמור לדעת, מול תקלת רשת או כניסה בתחנת Windows, איזו פקודה עונה על כל שאלה (יש Link? יש כתובת נכונה? יש שכן? יש דרך? השם נפתר? השירות מקבל חיבור? התחנה מוכרת בדומיין?), להצביע בפלט על השדה שעונה עליה, ולהגיד מה התוצאה עדיין לא מוכיחה.

הכנה

- פתחו את המצגת ב-Chrome או ב-Edge במחשב שמחובר למקרן, ועברו על כל השקופיות. המצגת לא צריכה חיבור לרשת.
- הניווט: חצים, רווח או PageUp/PageDown. Home ו-End קופצים לשקופית הראשונה והאחרונה. N פותח את הערות המדריך, F מסך מלא, A מכבה ומדליק אנימציות (כשהן כבויות מופיע תג ״אנימציות כבויות״).
- אין במצגת שדות הקלדה. בסימולטור (שקופית 13) עובדים רק בלחיצות עכבר. גם כשהפוקוס על כפתור של הסימולטור, רווח וחצים מעבירים שקופית, ולכן לא לוחצים רווח כדי להפעיל כפתור.
- מספרי השקופיות במדריך הם המספרים שבמונה בתחתית המסך (״2 / 14״). המספר הקטן שליד הכותרת בכל שקופית נמוך באחד, כי שקופית השער לא ממוספרת.
- ודאו שהפקודות זמינות בגרסת Windows של התחנה שבה תשתמשו. ב-Windows 10 ו-11 כל הפקודות בשיעור מובנות. הכירו את החלון הגרפי המקביל לכל cmdlet של PowerShell, למקרה שהוא חסר בתחנה ישנה.
- אם אתם רוצים להראות פלט אמיתי ליד השקופיות, הכינו תחנת תרגול והשתמשו בכתובות ובשמות לדוגמה, בלי נתונים רגישים. פלט של תחנה אמיתית מציג כתובות, שמות שרתים ושמות משתמשים.

היקף השיעור

עובדים רק בכלים המובנים ב-Windows, ומכל פלט קוראים את השדה שעונה על השאלה. הגדרות מערכת לא משנים בשיעור: לא משחררים כתובת, לא מוחקים מטמון, לא מצרפים תחנה לדומיין. תיקון ערוץ אמון, שמופיע בסיכום הסימולטור, דורש הרשאות דומיין ונעשה בידי הגורם המוסמך לפי נוהל היחידה. השיעור גם לא נכנס ל-UDP ולא למצבי TCP מעבר ל-LISTENING ו-ESTABLISHED.

חלוקת זמן מוצעת

חלק	שקופיות	מה מעבירים	דקות
פתיחה	1-2	בחירת כלי לפי שאלה	5
מתאם וכתובת	3-4	ipconfig ו-Get-NetAdapter	10

שכנות ומסלול	5-7	ping, arp, route ו-tracert	12
שם ופורט	8-9	Resolve-DnsName ו-Test-NetConnection	10
מצב חיבורים	10	netstat / Get-NetTCPConnection	6
Domain ואמון	11-13	w32tm, nltest, klist, תעודות ואירועים, סימולטור התחקור	9
סגירה	14	שרשרת ראיות	3

סך הכול 55 דקות. תשע הדקות של החלק האחרון צפופות: הסימולטור לבדו, כשנותנים לכיתה לבחור בכל שלב, לוקח בדרך כלל יותר. אם אין זמן, עצרו בסימולטור אחרי שלב 4, כמו שכתוב בהערות השקופית, ובקשו מהחניכים להציע את הבדיקות שנשארו. אם יש זמן, עדיף לתת לסימולטור את מלוא הזמן ולקצר את שקופיות 10 ו-12.

שאלה לפתיחה

איזו פקודה עונה על השאלה ״האם Windows רואה Link״? (Get-NetAdapter, שקופית 3). שאלו אותה לפני שמראים את שקופית 2, ואחר כך בדקו את התשובות מול המפה.

פתיחה: כלי לכל שאלה

שקופית 1 · כלי Windows: מה בודקים בכל פקודה

בשער מופיעה רשימת השאלות של השיעור: Link, כתובת, שכנות, מסלול, DNS, שירות, Domain ואמון. ההערה לשקופית קובעת את הקו לכל השיעור: בכל כלי מראים את השדה שעונה על השאלה, ואומרים מה התוצאה עדיין לא מוכיחה. את רשימת הפקודות לא צריך לשנן. טכנאי שזוכר איזו שאלה כל פקודה עונה עליה ימצא את התחביר בעזרה (Get-Help או ?/).

שקופית 2 · איזה כלי עונה על איזו שאלה

על המסך: שרשרת של שבעה עיגולים, משמאל לימין לפי סדר הבדיקה, ומתחת לכל אחד הכלי שעונה עליו: Link (Get-NetAdapter, כתובת (ipconfig /all), שכן (arp -a), מסלול (route / tracert), שם (Resolve-DnsName), שירות (Test-NetConnection) וזהות (w32tm / klist).

רקע. הסדר בשרשרת הוא סדר התלות. כדי לקבל כתובת צריך Link. כדי להגיע לשער צריך כתובת ומסכה נכונות, ואחר כך שהתחנה תמצא את ה-MAC של השער. כדי להגיע לשרת ברשת אחרת צריך מסלול. כדי לפנות לשרת בשם צריך DNS. רק אחרי כל אלה יש טעם לבדוק אם השירות מקבל חיבור, ובסוף אם התחנה והמשתמש מוכרים בדומיין. כל כלי בודק חוליה אחת, ותוצאה תקינה בחוליה אחת לא אומרת כלום על החוליות שאחריה.

כותרת המשנה אומרת שבוחרים את הבדיקה הבאה לפי מה שהכלי הקודם הראה, ושאר הבדיקות עדיין נדרשות. בפועל זה אומר: לא מריצים את כל הפקודות בכל תקלה, ולא מדלגים לסוף. אם המשתמש מתלונן שאתר פנימי לא נפתח אבל שאר הרשת עובדת, אין טעם להתחיל מ-Get-NetAdapter. מתחילים מהשם ומהשירות. אם התחנה לא מגיעה לשום דבר, מתחילים מה-Link. מריצים את הכלי שעונה על השאלה הפתוחה ומשווים את התוצאה לצפוי.

איך להעביר. עברו על המפה עיגול אחרי עיגול, ובכל אחד בקשו מהחניכים לנסח את השאלה שהכלי עונה עליה, במילים שלהם. ההערה לשקופית מזכירה שלא צריך להריץ כל כלי בכיתה. חשוב שהחניכים יידעו מה כל כלי מוכיח.

במפה לא מופיעים שני כלים שנלמדים בהמשך: מצב החיבורים (netstat, שקופית 10), ששייך לשאלת השירות, ו-Certificate Viewer ו-Event Viewer (שקופית 12), ששייכים לשאלת האמון.

שאלה	תשובה
למה לא להריץ את כל הפקודות כל פעם, ליתר ביטחון?	כי פלט שלא מחפשים בו תשובה לשאלה מסוימת לא מוסיף מידע, רק רעש. מי שמריץ עשר פקודות בלי שאלה נוטה להיתפס בשורה שנראית חשודה ולא קשורה לתקלה.

המתאם והזהות המקומית

שקופית 3 · Get-NetAdapter מראה אם Windows מזהה מתאם, Link וקצב קישור

על המסך: פלט מקוצר של Get-NetAdapter עם שני מתאמים: Ethernet0 במצב Up בקצב 1 Gbps, ו-Ethernet1 במצב Disconnected. מתחת: הפלט מתאר רק את המתאם בתחנה, והרשת שמעבר לכבל לא נבדקת.

רקע. Get-NetAdapter הוא cmdlet של PowerShell שמציג את מתאמי הרשת ש-Windows מכיר. בפלט האמיתי העמודות הן Name, InterfaceDescription (שם הדגם והיצרן), ifIndex, Status, MacAddress ו-LinkSpeed. בשקופית הוצגו רק ארבע מהן.

- **Status** הוא מצב הקישור. Up אומר שהכרטיס מזהה אות חשמלי או אופטי מהצד השני וסיכם איתו קצב. Disconnected אומר שאין Link: כבל מנותק, כבל פגום, יציאה כבויה במתג. Disabled אומר שמישהו השבית את המתאם ב-Windows. Not Present מופיע כשהמתאם הוגדר אבל החומרה לא נמצאת.
- **LinkSpeed** הוא הקצב שסוכם מול המתג. קצב נמוך מהצפוי הוא ממצא: כרטיס ומתג של 1 Gbps שסיכמו על 100 Mbps מצביעים לרוב על כבל או מחבר. קישור של 1 Gbps בנחושת משתמש בכל ארבעת זוגות הכבל, וכשזוג אחד פגום הכרטיסים יכולים לרדת ל-100 Mbps, שמסתפק בשני זוגות.
- **MacAddress** הוא הכתובת הפיזית של הכרטיס. נשתמש בה בשקופית 6 כשמשווים לטבלת ARP של תחנה אחרת, או לטבלת הכתובות במתג.

כותרת המשנה היא העיקר: בלי Up אין טעם להמשיך, אבל Up לא מוכיח שהיציאה במתג משויכת ל-VLAN הנכון, שהכתובת נכונה או שיש מסלול. כבל תקין ליציאה ב-VLAN הלא נכון נותן Up מושלם, ותחנה שלא מגיעה לשום דבר.

מתאמים וירטואליים. בתחנה יכולים להופיע מתאמים שאינם כרטיס פיזי: מתאם VPN, מתאם וירטואלי של Hyper-V או של תוכנת מכונות וירטואליות, מתאם Bluetooth. חלקם מופיעים במצב Up גם בלי שום כבל. לפני שמסיקים משהו, בודקים שמסתכלים על המתאם שהתחנה באמת משתמשת בו. Get-NetAdapter -Physical מציג רק מתאמים פיזיים, ושדה InterfaceDescription עוזר לזהות איזה מהם הוא כרטיס הרשת שמחובר לשקע.

החלופה הגרפית. ההערה לשקופית: אם הכלי לא זמין בגרסת PowerShell המקומית, בודקים את אותם שדות בחלון Network Connections (ncpa.cpl). שם רואים את המתאמים ואת המצב שלהם, ובחלון Status של מתאם רואים את הקצב (Speed). ב-Windows 10 ו-11 הכלי קיים תמיד. החלופה רלוונטית בעיקר לתחנות ישנות.

שאלה	תשובה
יש Up אבל אין רשת. מה הלאה?	הכתובת (שקופית 4) והשער (שקופיות 5 ו-6). אם הכתובת היא 169.254.x.x בתחנה שאמורה לקבל DHCP, התחנה לא קיבלה תשובה משרת DHCP, ובדרך כלל זה מצביע על VLAN או על היציאה במתג.

טעות נפוצה

לראות Up ולסמן "הרשת תקינה". Up אומר רק שיש קישור פיזי בין הכרטיס ליציאה במתג. הוא לא מוכיח VLAN, IP או דרך. טעות שנייה: לבדוק את המתאם הלא נכון, למשל מתאם וירטואלי שנמצא תמיד ב-Up.

שקופית 4 · ipconfig /all מציג את הזהות המקומית של התחנה ברשת

על המסך: חמישה שדות מהפלט: IPv4 Address 10.20.30.15, Subnet Mask 255.255.255.0, Default Gateway 10.20.30.1, DNS Servers 10.20.10.10 ו-DHCP Enabled No.

רקע. ipconfig בלי פרמטרים מציג רק כתובת, מסכה ושער. עם /all הוא מציג לכל מתאם גם את שם המתאם והדגם (Description), את הכתובת הפיזית (Physical Address), אם הכתובת מגיעה מ-DHCP (DHCP Enabled), את שרתי ה-DNS, ובראש הפלט את שם התחנה (Host Name) ואת סיומת הדומיין (Primary Dns Suffix). כשהכתובת מגיעה מ-DHCP מופיעים גם שרת ה-DHCP ומועדי החכירה.

כל שדה עונה על שאלה אחרת, וכל אחד יכול להיות שגוי בלי שהאחרים ייראו חשודים:

- **IPv4 Address.** ליד הכתובת כתוב (Preferred) כשהכל תקין. (Duplicate) אומר ש-Windows זיהה תחנה אחרת עם אותה כתובת. כתובת 169.254.x.x היא כתובת ש-Windows נתן לעצמו כשלא קיבל תשובה מ-DHCP.
- **Subnet Mask.** המסכה קובעת אילו כתובות התחנה מחשיבה "מקומיות" ושולחת אליהן ישירות, ואילו היא שולחת לשער. מסכה רחבה מדי (למשל 255.255.0.0 במקום 255.255.255.0) גורמת לתחנה לחשוב ששרת ברשת אחרת נמצא לידה. היא מחפשת אותו ב-ARP במקום לשלוח לשער, ואף אחד לא עונה. כל השאר נראה תקין.
- **Default Gateway.** לאן נשלחת כל תעבורה ליעד שאינו מקומי. שער שגוי, או שער שלא נמצא בתוך הרשת שהמסכה מגדירה, משאיר את התחנה מחוברת רק לרשת המקומית.
- **DNS Servers.** את מי התחנה שואלת על שמות. בתחנה בדומיין זה צריך להיות שרת DNS של הדומיין. תחנה שמצביעה על שרת DNS אחר תפתור אולי שמות כלליים, אבל לא תמצא את בקר הדומיין. אם רשומים כמה שרתים, השני נשאל רק כשהראשון לא עונה בכלל. אם הראשון עונה "אין שם כזה", התחנה לא ממשיכה לשני.
- **DHCP Enabled.** בסביבה סטטית הערך צריך להיות No. Yes בתחנה שאמורה להיות סטטית אומר שמישהו שינה את ההגדרה, והכתובת עלולה להשתנות.

כותרת המשנה וההערה: בסביבה סטטית משווים כל שדה לתכנון, או לתחנה תקינה באותה רשת, גם כשהערכים נראים סבירים. אין להסתפק בכתובת IP. השוו גם מסכה, Gateway, DNS, DHCP Enabled ושם המתאם. כתובת סבירה במבט ראשון יכולה להיות כתובת של תחנה אחרת, וערך DNS סביר יכול להיות של שרת שהוצא משימוש.

בשטח. ב-PowerShell יש מקבילה קצרה, Get-NetIPConfiguration, שמציגה לכל מתאם את הכתובת, השער וה-DNS. בחלון הגרפי: Network Connections, לחיצה כפולה על המתאם ואז Details. מקור הערכים הנכונים (תכנון הכתובות של הרשת) נקבע לפי נוהל היחידה.

שאלה	תשובה
למה יש בפלט כל כך הרבה מתאמים?	ipconfig מציג כל מתאם ש-Windows מכיר, כולל וירטואליים, VPN ו-Bluetooth. מתאם שאין לו Link מופיע עם Media disconnected. מחפשים את המתאם שמחובר בפועל.

tracert-ו route ,ARP ,Ping

שקופית 5 · Ping בודק תשובת ICMP ומודד RTT

על המסך: ארבעה יעדים ברצף: 127.0.0.1 (TCP/IP מקומי), כתובת התחנה (ממשק מקומי), Gateway (הרשת המקומית) ויעד מרוחק (מסלול מקצה לקצה). בתחתית: כישלון לא מוכיח שהיעד כבוי, כי ICMP עשוי להיות מסונן.

רקע. ping שולח הודעת ICMP מסוג Echo Request ומחכה ל-Echo Reply. ב-Windows ברירת המחדל היא ארבע בקשות של 32 בתים, וכל בקשה מחכה לתשובה עד ארבע שניות. בכל שורת תשובה רואים bytes, time (זמן הלוך-חזור, RTT) ו-TTL. בסוף מופיע סיכום: כמה נשלחו, כמה התקבלו ואחוז האובדן, ואחריו RTT מינימלי, מקסימלי וממוצע.

TTL בתשובה הוא הערך שהיעד שלח, פחות מספר הנתבים שבדרך. Windows שולח בדרך כלל 128, Linux בדרך כלל 64, ורכיבי רשת רבים 255. תשובה עם TTL של 128 מתחנה ברשת המקומית מתאימה לשרת Windows שעונה ישירות. TTL של 126 מאותו שרת אומר שיש שני נתבים בדרך.

הסדרה שבשקופית. כל יעד מוסיף חוליה אחת:

- 127.0.0.1 לא יוצא מהתחנה בכלל. הצלחה מראה שרכיב ה-TCP/IP של Windows עובד. כישלון כאן נדיר מאוד ומצביע על בעיה בתחנה עצמה.
- הכתובת של התחנה עצמה גם היא לא יוצאת לרשת. הצלחה מראה שהכתובת מוגדרת ופעילה על המתאם.
- ה-Gateway הוא היעד הראשון שבאמת יוצא לכבל. הצלחה מראה שיש Link, שהכתובת והמסכה מאפשרות לדבר עם השער, ושהשער נמצא ב-ARP (שקופית 6).
- יעד מרוחק בודק גם את המסלול שמעבר לשער, ואת הדרך חזרה.

ההערה לשקופית: הסדרה ממחישה מה בודק כל יעד, ולא חייבים להריץ את כולה בכל תקלה. אם השער עונה, אין טעם לבדוק את 127.0.0.1.

הודעות השגיאה אומרות דברים שונים.

הודעה	מה היא אומרת
Request timed out.	לא חזרה תשובה בזמן. היעד כבוי, מסנן ICMP, או שהתשובה לא מוצאת דרך חזרה.
Destination host unreachable.	מישהו בדרך הודיע שאין לו איך להגיע. אם התשובה מגיעה מהכתובת של התחנה עצמה, התחנה לא מצאה את היעד ב-ARP ברשת המקומית, או שאין לה מסלול אליו. אם היא מגיעה מכתובת של נתב, הנתב הוא שלא הצליח.
TTL expired in transit.	החבילה עברה יותר מדי נתבים. בדרך כלל לולאת ניתוב.
Ping request could not find host	השם לא נפתר. זו תקלת DNS (שקופית 8), ושום חבילת ICMP לא נשלחה.

מה ping לא בודק. כותרת המשנה: הוא לא בודק זמינות של יישום. שרת יכול לענות ל-ping כשהשירות עליו נפל, ושרת יכול לא לענות ל-ping כשהשירות עובד מצוין. בהגדרות ברירת המחדל של חומת האש של Windows, תחנת קצה בדרך כלל לא עונה ל-ping נכנס, אלא אם הופעל הכלל המתאים, לרוב במדיניות. שרתים, נתבים וחומות אש רבים מסננים ICMP בכוונה.

בשטח. ping -t שולח בלי הפסקה עד Ctrl+C, ומתאים למעקב בזמן שמחברים כבל או מזיזים ציוד. ping -n 20 שולח 20 בקשות, ומראה אובדן לסירוגין שארבע בקשות לא יתפסו. ping -4 מכריח IPv4 כשהשם נפתר גם לכתובת IPv6.

שאלה	תשובה
Ping לשרת נכשל. השרת כבוי?	לא בהכרח. בודקים אם ping לשער עובד, ואם השירות עצמו מקבל חיבור (Test-NetConnection עם הפורט, שקופית 9).

טעות נפוצה

לקרוא רק את שורת הסיכום. כשהתחנה עצמה עונה Destination host unreachable, Windows סופר את זה כתשובה שהתקבלה, ושורת הסיכום מראה (0% loss) Lost = 0 למרות שהיעד לא ענה אף פעם. קוראים את שורות התשובה ובודקים מאיזו כתובת הגיעה התשובה.

שקופית 6 · arp -a מראה איזה MAC משויך ל-IP המקומי או ל-Gateway

על המסך: פלט של arp -a עם שתי רשומות: השער 10.20.30.1 עם MAC 00-15-5d-01-02-03, ותחנה 10.20.30.80 עם MAC 34-7a-60-aa-bb-cc, שתיהן dynamic. בתחתית: רשומה חסרה או MAC לא צפוי מכוונים לבעיה מקומית, VLAN או שכנות.

רקע. ברשת מקומית (Ethernet או Wi-Fi) מסגרות נשלחות לפי כתובת MAC. כשהתחנה צריכה לשלוח חבילה לכתובת IP ברשת המקומית שלה, היא שואלת בשידור לכולם ״מי מחזיק את הכתובת הזו?״, והבעלים עונה עם ה-MAC שלו. זה ARP. התשובה נשמרת לזמן קצר בטבלה, כדי לא לשאול שוב לפני כל חבילה, ו-arp -a מציג את הטבלה הזו.

הפלט מחולק לפי ממשק. מעל כל קבוצה מופיעה שורה כמו Interface: 10.20.30.15 --- 0xb, ומתחתיה שלוש עמודות: Internet Address, Physical Address ו-Type. dynamic נלמד מתשובת ARP. static הוגדר ידנית, או שייך לכתובות שידור ו-multicast (למשל כתובת השידור של הרשת עם ff-ff-ff-ff-ff-ff), שמופיעות בטבלה תמיד ולא צריך להתייחס אליהן.

מה רואים ומה לא. בטבלה מופיעות רק כתובות מהרשת המקומית. תחנה שפונה לשרת ברשת אחרת שולחת את המסגרת ל-MAC של השער, ולכן הכתובת של השרת המרוחק לא תופיע ב-ARP לעולם. השער מעביר את החבילה הלאה, ובכל קפיצה נבנית מסגרת Ethernet חדשה עם כתובות MAC חדשות, בעוד שכתובות ה-IP של המקור והיעד נשארות כמו שהן.

הממצאים שכותרת המשנה מתכוונת אליהם:

- **רשומה חסרה לשער** אחרי שניסו לשלוח אליו. התחנה שאלה ואף אחד לא ענה: השער לא באותו VLAN, היציאה במתג משויכת לרשת אחרת, או שהמסכה שגויה. ב-PowerShell, Get-NetNeighbor מראה גם רשומות שנכשלו, עם מצב Unreachable, וזה ממצא ברור יותר מרשומה חסרה.
- **MAC לא צפוי.** אם ה-MAC של השער שונה מזה שבתחנה תקינה באותה רשת, מכשיר אחר עונה על הכתובת של השער: מכשיר עם הגדרה שגויה, או ניסיון התחזות.
- **כפילות.** אותו IP שה-MAC שלו מתחלף בין בדיקה לבדיקה מצביע על שני מכשירים עם אותה כתובת.

ההערה לשקופית: משווים למערכת תקינה או לתכנון. כתובת MAC לבדה אינה ״טובה״ או ״רעה״.

שימו לב

רשומת ARP חסרה לבדה אינה תקלה. ייתכן שעדיין לא נשלחה תעבורה ליעד, או שהרשומה פגה כי לא הייתה תעבורה זמן מה. קודם שולחים משהו ליעד (למשל ping), ורק אז בודקים את הטבלה.

שאלה	תשובה
למה השרת שאני עובד מולו לא מופיע בטבלה?	כי הוא ברשת אחרת. מה שמופיע הוא ה-MAC של השער, ודרכו עוברת כל התעבורה לשרת.

אפשר לדעת מ-MAC מי היצרן?	שלושת הבתים הראשונים מזהים בדרך כלל את יצרן הכרטיס. זה עוזר להבחין, למשל, בין מתג למחשב, אבל לא מוכיח זהות, כי אפשר לשנות MAC בתוכנה.

שקופית 7 · route print ו-tracert בודקים את המסלול

על המסך: שני חלונות. בשמאל route print עם שתי שורות: 10.20.40.0 דרך 10.20.30.2, ו-0.0.0.0 דרך 10.20.30.1, ושאלה: איזה שער ייבחר? בימין tracert 10.20.40.25 עם שלוש קפיצות: 10.20.30.1, 10.20.35.1, 10.20.40.25, ושאלה: היכן נראית התגובה האחרונה?

route print. הפקודה מציגה את טבלת הניתוב של התחנה: לאיזה שער התחנה תשלח, עוד לפני שיש תעבורה. בפלט האמיתי יש קודם רשימת ממשקים, ואחריה הטבלה של IPv4 (Active Routes) עם חמש עמודות: Network Destination, Netmask, Gateway, Interface ו-Metric. בשקופית הוצגו רק היעד והשער. On-link בעמודת Gateway אומר שהיעד ברשת המקומית ונשלח ישירות. בסוף מופיעים Persistent Routes, מסלולים שהוגדרו ידנית ונשמרים אחרי אתחול.

כשיש כמה שורות שמתאימות ליעד, נבחרת השורה הספציפית ביותר, זו עם המסכה הארוכה ביותר. רק כשיש שתי שורות באותו אורך מסכה, המטריקה הנמוכה מכריעה. 0.0.0.0 עם מסכה 0.0.0.0 מתאים לכל יעד, והוא ברירת המחדל: מה שלא מתאים לשום שורה אחרת יוצא לשער שלו.

התשובה לשאלה שבשקופית: יעד ב-10.20.40.x מתאים לשתי השורות, והשורה של 10.20.40.0 ספציפית יותר (בהנחה שהמסכה שלה 255.255.255.0, שלא מוצגת), ולכן התעבורה יוצאת ל-10.20.30.2. כל יעד אחר יוצא ל-10.20.30.1.

tracert. tracert מראה מי השיב לאורך הדרך. הוא שולח חבילות עם TTL שעולה בהדרגה: 1, אחר כך 2, וכן הלאה. כל נתב מוריד את ה-TTL באחד, ונתב שמקבל חבילה עם TTL שנגמר זורק אותה ושולח בחזרה הודעת ICMP ״Time Exceeded״. מהכתובת של ההודעה הזו tracert יודע מי הנתב בכל קפיצה. ב-Windows החבילות הן ICMP Echo, שלוש לכל קפיצה, עד 30 קפיצות כברירת מחדל, ולכן בכל שורה יש שלושה זמנים. tracert של Windows גם מנסה לתרגם כל כתובת לשם, וזה מאט אותו. tracert -d מדלג על התרגום.

השאלה ״היכן נראית התגובה האחרונה?״ היא העיקר. כשהמסלול נקטע, tracert מראה עד איפה הגיעו תשובות, והקפיצה האחרונה שענתה מסמנת את האזור שבו כדאי לחפש. כשיש רק כוכביות מקפיצה מסוימת ועד הסוף, זה מצביע על נקודה שאחריה החבילות לא עוברות, או שהיעד מסנן ICMP.

ההערה לשקופית: כוכבית ב-tracert אינה בהכרח תקלה. יש ציוד שאינו משיב, או מגביל את קצב התשובות שלו, אבל ממשיך להעביר תעבורה. אם אחרי שורה של כוכביות מופיעות קפיצות שעונות, החבילות עברו דרך הציוד השקט.

> **שני החלונות לא מתאימים זה לזה**
>
> לפי טבלת הניתוב שבשקופית, הקפיצה הראשונה ל-10.20.40.25 צריכה להיות 10.20.30.2, וב-tracert היא 10.20.30.1. אם חניך שם לב לזה, הוא צודק. אפשר להשתמש בזה: שאלו מה הייתם חושדים אם הייתם רואים את שני הפלטים באותה תחנה. התשובה: ה-tracert לא יצא לפי הטבלה שראיתם, למשל כי הטבלה נלקחה מתחנה אחרת או כי המסלול השתנה בין שתי הבדיקות. בתחנה אחת ובאותו רגע, הקפיצה הראשונה ב-tracert אמורה להיות השער שנבחר בטבלה (אם השער לא משיב, יופיעו בה כוכביות).

בשטח. ב-PowerShell, Find-NetRoute -RemoteIPAddress 10.20.40.25 מראה איזה מסלול וכתובת מקור Windows יבחר ליעד מסוים, בלי לחשב בראש. pathping משלב tracert עם סטטיסטיקת אובדן לכל קפיצה, אבל לוקח כמה דקות.

שאלה	תשובה
הקפיצה השנייה מראה כוכביות, אבל היעד עונה. יש בעיה?	לא. הנתב בקפיצה השנייה לא שולח הודעות Time Exceeded, אבל מעביר תעבורה. אם היעד עונה, הדרך תקינה.
למה הזמן בקפיצה באמצע גבוה מהזמן ליעד?	נתבים מטפלים בתשובות ICMP בעדיפות נמוכה. זמן גבוה בקפיצה אחת, כשהקפיצות שאחריה מהירות, לא מעיד על עומס בדרך.

שם, פורט ומצב חיבורים

שקופית 8 · Resolve-DnsName או nslookup בודקים את ה-DNS בנפרד מהשירות

על המסך: Resolve-DnsName server.unit ושורת תשובה: השם, סוג רשומה A, TTL של 300 והכתובת 10.20.40.25. מתחת: בודקים גם איזה שם נשאל ומה נשמר ב-Cache, עם הפקודה ipconfig /displaydns.

רקע. כשמשתמש פונה לשרת בשם, התחנה צריכה קודם להפוך את השם לכתובת. אם השלב הזה נכשל, או מחזיר כתובת שגויה, שום בדיקה של השירות לא תעזור. לכן בודקים את השם לבד: כשהמשתמש פונה בשם, בודקים קודם את פתרון השם, ורק אחר כך את השירות בכתובת שהוחזרה.

בתשובה מחפשים:

- **איזה שם נשאל.** כשמשתמש מקליד שם קצר, כמו server, Windows משלים אותו בסיומת הדומיין (למשל server.unit.local) ושואל על השם המלא. עמודת Name בתשובה מראה על מה בדיוק נשאל.
- **סוג הרשומה.** A היא כתובת IPv4, AAAA כתובת IPv6, CNAME שם חלופי שמפנה לשם אחר, PTR תרגום הפוך מכתובת לשם, ו-SRV רשומה שמפרסמת שירות (איזה שרת ובאיזה פורט). בסימולטור, בשקופית 13, רואים איך תחנה מוצאת את בקר הדומיין דרך רשומת SRV.
- **הכתובת שהוחזרה.** משווים לכתובת המתוכננת של השרת. כתובת ישנה מצביעה על רשומה שלא עודכנה או על מטמון.
- **מי ענה.** כותרת המשנה מבקשת לבדוק את השרת שענה. כאן יש הבדל בין שני הכלים: nslookup מדפיס בראש הפלט Server: ו-Address: של שרת ה-DNS שענה, ו-Resolve-DnsName לא מציג את זה. כדי לשאול שרת מסוים ב-Resolve-DnsName מוסיפים 10.20.4.10 -Server, וב-nslookup כותבים את השרת אחרי השם: nslookup server.unit 10.20.4.10.

TTL והמטמון. TTL הוא מספר השניות שמותר לשמור את התשובה. התחנה שומרת תשובות במטמון (Cache) של Windows, ובפעם הבאה עונה מהמטמון בלי לשאול. אם שואלים שוב תוך זמן קצר, ה-TTL שמוצג קטן יותר, כי זו אותה תשובה שמחכה לפוג. ipconfig /displaydns מציג את המטמון, ו-Get-DnsClientCache הוא המקבילה ב-PowerShell. גם תשובה שלילית ("אין שם כזה") נשמרת זמן מה, ולכן תחנה יכולה להמשיך "לא למצוא" שרת דקות אחרי שהרשומה תוקנה. את המטמון מוחקים (ipconfig /flushdns) רק בשטח, כשיש לזה סיבה.

ההבדל בין שני הכלים. Resolve-DnsName עובר דרך רכיב פתרון השמות של Windows, כמו כל תוכנה בתחנה: הוא מתחשב בקובץ hosts ובמטמון, ואם DNS לא עונה הוא עשוי לנסות גם שיטות מקומיות (LLMNR ו-NetBIOS). nslookup- מגביל אותו ל-DnsOnly. הוא כלי נפרד ששולח שאילתה ישירות לשרת DNS, בלי קובץ hosts ובלי המטמון של Windows. לכן כששני הכלים נותנים תשובות שונות, זה ממצא: כנראה יש רשומה בקובץ hosts (C:\Windows\System32\drivers\etc\hosts) או תשובה ישנה במטמון.

ההערה לשקופית: כאשר צריך לבדוק SRV, מציינים את סוג הרשומה במפורש (Type SRV-). אין צורך ללמד תחביר מתקדם.

שאלה	תשובה
nslookup מראה UnKnown :Server. ה-DNS לא תקין?	לא בהכרח. nslookup מנסה להציג את שם השרת לפי הכתובת שלו, ואם אין לשרת רשומת PTR הוא כותב UnKnown. התשובה עצמה מופיעה מתחת, וזה מה שבודקים.
מה זה Non-authoritative answer?	שהשרת שענה אינו השרת שאחראי על השם, והוא ענה ממה ששמר אצלו. זה המצב הרגיל ואינו תקלה.

השם נפתר, אבל לכתובת אחרת מזו שבתכנון. מה עושים?	בודקים אם התשובה הגיעה מהמטמון או מקובץ hosts (משווים ל-nslookup מול השרת), ומדווחים על הרשומה לגורם שמנהל את ה-DNS.

טעות נפוצה

לבדוק DNS עם שם של אתר כללי ולהסיק שה-DNS תקין. שרת DNS יכול לפתור שמות כלליים ולא להכיר את שמות הדומיין, ולהפך. בודקים בדיוק את השם שנכשל.

שקופית 9 · Test-NetConnection בודק אם ניתן לפתוח חיבור TCP לפורט מסוים

על המסך: Test-NetConnection server.unit -Port 443 וארבעה שדות: ComputerName, RemoteAddress 10.20.40.25, RemotePort 443 ו-TcpTestSucceeded True. מתחת: מוכיח פתיחת TCP ליעד ולפורט, ותקינות היישום עצמו לא נבדקת.

רקע. שירות ברשת מאזין לפורט: שרת אינטרנט ל-443 (HTTPS) או 80 (HTTP), שיתוף קבצים ל-445, שולחן עבודה מרוחק ל-3389, LDAP ל-389. כדי לדבר עם שירות בפרוטוקול TCP, הלקוח פותח חיבור בלחיצת יד של שלוש הודעות: הלקוח שולח SYN, השרת עונה SYN-ACK אם יש שירות שמאזין, והלקוח מאשר ב-ACK. Test-NetConnection עם Port- מנסה בדיוק את זה ומדווח אם הצליח.

כותרת המשנה אומרת שהתוצאה מרכזת שם, כתובת, מסלול ופורט בבדיקה אחת. בפלט המלא מופיעים: ComputerName (השם שנבדק), RemoteAddress (הכתובת שהשם נפתר אליה, כך שרואים גם את תוצאת ה-DNS), RemotePort, InterfaceAlias ו-SourceAddress (דרך איזה מתאם ומאיזו כתובת יצא החיבור, וזה החלק של "מסלול"), ובסוף TcpTestSucceeded. בשקופית הוצגו ארבעה שדות מתוכם.

כשהבדיקה נכשלת מופיעה אזהרה צהובה (WARNING: TCP connect to ... failed), ו-Test-NetConnection מוסיף מעצמו בדיקת ping: PingSucceeded ו-PingReplyDetails. הצירוף שימושי. ping מצליח ופורט נכשל אומר שיש דרך לשרת אבל השירות לא מאזין, או שחומת אש חוסמת את הפורט. שניהם נכשלים: אולי אין דרך בכלל, ואולי פשוט שניהם מסוננים. אם השם עצמו לא נפתר, מופיעה אזהרה על Name resolution, וזו כבר שאלה של שקופית 8.

בלי Port- הפקודה עושה רק ping ומציגה PingSucceeded. זה מקור לבלבול: מי ששוכח את הפורט בודק רק ICMP.

מה לא הוכח

TcpTestSucceeded : True מוכיח שנפתח חיבור TCP ליעד ולפורט. הוא לא מוכיח שהיישום החזיר תוכן תקין. שרת אינטרנט יכול לקבל חיבור ולהחזיר דף שגיאה, להציג שגיאת תעודה או לדחות את המשתמש בהרשאות. את אלה בודקים ביישום עצמו, למשל בדפדפן.

ההערה לשקופית: במערכת שבה השירות אינו TCP, משתמשים בכלי הייעודי. UDP אינו מוקד השיעור. Test-NetConnection לא בודק UDP כלל, ובפרוטוקול בלי לחיצת יד אין דרך פשוטה לדעת מבחוץ אם מישהו מאזין.

בשטח. הכינוי הקצר הוא tnc. InformationLevel Quiet- מחזיר רק True או False, נוח לבדיקה מהירה. כשחיבור נכשל מהר, בדרך כלל משהו דחה אותו באופן פעיל (השרת בלי שירות בפורט, או חומת אש שמחזירה דחייה). כשהוא נכשל רק אחרי המתנה ארוכה, בדרך כלל משהו בדרך זורק את החבילות בשקט.

שאלה	תשובה
True, ובכל זאת האתר לא עולה. איפה ממשיכים?	ביישום: הודעת השגיאה בדפדפן, התעודה (שקופית 12), ההרשאות של המשתמש. הרשת עד הפורט כבר נבדקה.
אפשר לבדוק כך פורט בתחנה שלי?	כן, Test-NetConnection localhost -Port 443 בודק אם משהו מאזין מקומית. כדי לדעת מי מאזין, עוברים לשקופית הבאה.

שקופית 10 · Get-NetTCPConnection או netstat מראים מי מאזין ומי מחובר

על המסך: פלט של Get-NetTCPConnection עם שתי שורות: 0.0.0.0 פורט 443 במצב Listen, ו-10.20.30.15 פורט 52144 מול 10.20.40.25 במצב Established. בתחתית: LISTENING אומר ששירות מחכה, ESTABLISHED אומר שנוצר חיבור.

רקע. Test-NetConnection בודק מבחוץ. כאן מסתכלים מבפנים, על המחשב עצמו: אילו פורטים פתוחים להאזנה, ואילו חיבורים קיימים כרגע. שני הכלים מציגים את אותו מידע. netstat ותיק וקיים בכל גרסה, ו-Get-NetTCPConnection הוא ה-cmdlet של PowerShell. שימו לב לכתיב: netstat כותב LISTENING ו-ESTABLISHED, וב-PowerShell אותם מצבים נקראים Listen ו-Established. בשקופית מופיעים שני הכתיבים.

כותרת המשנה מתארת את שני המצבים שחשובים בשיעור:

- **LISTENING** מופיע בצד השרת: שירות פתח פורט ומחכה לחיבורים. לרשומה כזו אין צד שני, ולכן הכתובת המרוחקת היא 0.0.0.0. הכתובת המקומית חשובה: 0.0.0.0 אומר שהשירות מאזין בכל הכתובות של המחשב, ו-127.0.0.1 אומר שהוא מאזין רק למחשב עצמו, ומבחוץ אי אפשר להגיע אליו. שירות שמאזין רק ב-127.0.0.1 יעבוד בבדיקה מקומית וייכשל בכל בדיקה מתחנה אחרת.
- **ESTABLISHED** הוא חיבור שכבר נוצר, עם הכתובת והפורט של הצד השני. בשורה השנייה בשקופית התחנה 10.20.30.15 מחוברת לשרת 10.20.40.25. הפורט המקומי, 52144, נבחר אוטומטית מטווח של פורטים זמניים, שב-Windows מתחיל בדרך כלל ב-49152. הפורט של השירות הוא הפורט בצד של השרת.

PID ותהליך. ההערה לשקופית: במידת הצורך מקשרים PID לתהליך. כל שורה שייכת לתהליך במחשב. netstat -ano מציג את כל החיבורים (a), בכתובות ובמספרים בלי תרגום לשמות (n), עם מספר התהליך (o). ב-PowerShell העמודה נקראת OwningProcess. אחרי שיש מספר, Get-Process -Id 1234 או tasklist /fi "PID eq 1234" מראים איזו תוכנה זו. כך עונים על שאלות כמו ״מי תופס את פורט 443 בשרת הזה?״. הפקודה netstat -ano | findstr :443 מסננת את הפלט לפורט אחד.

ההערה גם אומרת שאין צורך להעמיק במצבי TCP האחרים. אם חניך שואל: TIME_WAIT הוא חיבור שנסגר ונשמר עוד זמן קצר, ומופיע בכמויות גם במחשב תקין. SYN_SENT שנשאר זמן רב בצד הלקוח אומר שהלקוח שלח בקשה ולא קיבל תשובה, וזה מתאים לפורט מסונן בדרך.

החלופה הגרפית. Resource Monitor (resmon), בלשונית Network, מציג Listening Ports ו-TCP Connections עם שם התהליך.

שאלה	תשובה
השירות רץ, אבל הפורט לא מופיע ב-LISTENING. מה זה אומר?	שהשירות לא פתח את הפורט שציפינו לו: הוא לא עלה עד הסוף, הוגדר לפורט אחר, או שתוכנה אחרת כבר תופסת את הפורט. בודקים לפי PID מי מאזין לפורט.
למה יש כל כך הרבה שורות?	כל תוכנה שמדברת ברשת פותחת חיבורים. מסננים לפי פורט או לפי כתובת, למשל Get-NetTCPConnection -LocalPort 443.

Domain, זמן, תעודה ואירועים

שקופית 11 · בדיקות Kerberos: זמן, איתור DC וכרטיסים

על המסך: שלושה כרטיסים: w32tm /query /status (זמן ומקור סנכרון, משווים לשעה של ה-DC), nltest /dsgetdc:domain (איתור Domain Controller: איזה DC נמצא, אם בכלל), ו-klist (כרטיסי Kerberos: יש כרטיסים בתוקף?).

רקע. כשמשתמש רשתי נכנס לתחנה בדומיין, התחנה מאמתת אותו מול בקר דומיין (DC) בפרוטוקול Kerberos. המשתמש מקבל "כרטיס" ראשי (TGT), ובכל פעם שהוא ניגש לשירות ברשת, כמו שיתוף קבצים, התחנה מבקשת בעזרתו כרטיס לאותו שירות. כדי שזה יעבוד צריכים להתקיים כמה תנאים, וכל אחד מהם נבדק בכלי אחר:

חוליה	מה היא נותנת	איך בודקים
DNS	איתור DC: התחנה מוצאת את בקר הדומיין לפי רשומות SRV ב-DNS	nltest /dsgetdc, או Resolve-DnsName על רשומת ה-SRV
זמן	תוקף הכרטיס: כרטיסים כוללים זמנים, וההפרש בין התחנה ל-DC חייב להיות קטן	w32tm
Kerberos	אימות: המשתמש והתחנה מוכחים מול ה-DC	klist
הרשאה	קבוצות: מה המשתמש רשאי לעשות אחרי שאומת	whoami /groups, ו-gpresult למדיניות

w32tm. Kerberos דוחה בקשות כשההפרש בין שעון התחנה לשעון ה-DC גדול מחמש דקות (ברירת המחדל במדיניות הדומיין). הסיבה: כל בקשה כוללת חותמת זמן, והגבלת ההפרש מונעת שימוש חוזר בבקשה שהוקלטה. התסמין נראה כמו בעיית סיסמה או הרשאה, ולכן כדאי לשלול את השעון מוקדם. w32tm /query /status מציג את מצב שירות הזמן של Windows. השורות החשובות:

- **Source**: מאיפה התחנה מקבלת זמן. בתחנה בדומיין זה צריך להיות שם של DC. Local CMOS Clock או Free-running System Clock אומרים שהתחנה לא מסונכרנת לשום מקור.
- **Last Successful Sync Time**: מתי הסתנכרנה בפעם האחרונה. תאריך ישן מאוד אומר שהסנכרון נכשל מאז, גם אם המקור רשום נכון.
- **Stratum**: כמה רמות התחנה רחוקה ממקור זמן מדויק. תחנה בדומיין נמצאת בדרך כלל רמה אחת מתחת ל-DC שלה.

הפקודה הזו לא מודדת ישירות את ההפרש מול ה-DC. כדי למדוד אותו מריצים w32tm /stripchart /computer:dc01.unit.local /samples:3 /dataonly, שמציג בכל שורה את ההפרש בשניות בין התחנה לשרת. ההערה בכרטיס, "משווים לשעה של ה-DC", מתכוונת לזה.

nltest /dsgetdc. במקום domain כותבים את שם הדומיין, למשל nltest /dsgetdc:unit.local. הפקודה מבקשת מ-Windows לאתר DC לדומיין, כמו שהתחנה עושה בכניסה. בהצלחה מופיעים בין השאר DC: \\dc01.unit.local, הכתובת שלו, שם הדומיין, האתר (Site) של ה-DC והאתר של התחנה, ושורת Flags עם התפקידים שלו. בכישלון מופיעה שורה כמו Getting DC name failed: Status = 1355 0x54b ERROR_NO_SUCH_DOMAIN, ובדרך כלל הסיבה היא DNS: התחנה שואלת שרת שלא מכיר את הדומיין. nltest הוא כלי בשורת הפקודה, ויש לו עוד בדיקות. בסימולטור, בשקופית 13, משתמשים ב-sc_verify/, שבודק את ערוץ האמון.

klist. מציג את כרטיסי ה-Kerberos של המשתמש המחובר. בראש הפלט Cached Tickets: (n), ואחריו לכל כרטיס: Client (המשתמש), Server (השירות שהכרטיס פותח), ו-Start Time, End Time, Renew Time. כרטיס שה-Server שלו

מתחיל ב-krbtgt/ הוא ה-TGT. כרטיסים אחרים פותחים שירותים, למשל cifs/fs01.unit.local לשיתוף קבצים. בברירת המחדל של הדומיין TGT תקף כעשר שעות וניתן לחידוש עד שבעה ימים. משתמש מקומי לא עובר דרך Kerberos, ולכן אצלו klist מציג אפס כרטיסים, וזה תקין.

ההערות לשקופית: gpresult יכול להצטרף כאשר הבעיה היא מדיניות (gpresult /r מציג אילו מדיניות הוחלו על המחשב ועל המשתמש). אל תציגו את כל פלט הפקודות. סמנו את השורה הרלוונטית.

שאלה	תשובה
הוסיפו משתמש לקבוצה, והוא עדיין בלי הרשאה. למה?	הקבוצות נכנסות לאסימון של המשתמש בכניסה. צריך להתנתק ולהיכנס מחדש. whoami /groups מראה את הקבוצות שבאסימון הנוכחי.
klist בחלון Administrator מראה רשימה ריקה. אין כרטיסים?	לא בהכרח. חלון שנפתח כמנהל רץ בהקשר כניסה נפרד, עם רשימת כרטיסים משלו. מריצים klist בחלון רגיל של המשתמש.
השעה בתחנה נכונה לפי השעון על הקיר. זה מספיק?	Kerberos משווה לשעה של ה-DC. גם אזור זמן שגוי משנה את השעה האמיתית של התחנה, אפילו כשהשעה שמוצגת נראית נכונה.

שקופית 12 · Certificate Viewer ו-Event Viewer

על המסך: בשמאל Certificate Viewer עם Subject server.unit, Issuer Unit-CA ו-Valid to 2027-07-15, ומתחת ״שם · זמן · שרשרת״. בימין Event Viewer: שעה, מקור ו-Event ID, הודעה מלאה, אירועים לפני ואחרי.

Certificate Viewer. זה החלון של Windows שנפתח כשלוחצים לחיצה כפולה על קובץ תעודה, כשפותחים תעודה מהדפדפן, או מתוך מאגרי התעודות (certmgr.msc לתעודות של המשתמש, certlm.msc לתעודות של המחשב). יש בו שלוש לשוניות. General מציג למי הונפקה (Issued to), מי הנפיק (Issued by) ותאריכי התוקף. Details מציג את כל השדות. Certification Path מציג את השרשרת עד תעודת השורש, ובתחתית שלה את מצב התעודה. בתעודה בודקים:

- **שם.** Subject הוא השם שהתעודה הונפקה לו. הדפדפנים היום בודקים את רשימת השמות שבשדה Subject Alternative Name בלשונית Details. השם שהמשתמש הקליד צריך להופיע שם בדיוק.
- **זמן.** Valid from ו-Valid to. התחנה משווה אותם לשעון שלה, ולכן שעון שגוי בתחנה נראה כמו תעודה שפג תוקפה.
- **מנפיק ושרשרת.** Issuer הוא מי שחתם על התעודה. בלשונית Certification Path רואים אם השרשרת מגיעה לשורש שהתחנה סומכת עליו. תחנה שחסר בה השורש הפנימי של הארגון לא תסמוך על אף תעודה שהוא הנפיק.

שגיאות תעודה וההסבר למה כל חוליה נבדקת מפורטים בשיעור 8 (הצפנה וזהות). כאן מספיק לדעת איפה רואים כל שדה.

Event Viewer (eventvwr.msc) מציג את יומני האירועים של Windows. תחת Windows Logs נמצאים System (שירותים, מנהלי התקן, רשת, שירות הזמן, Netlogon), Application (תוכנות), ו-Security (כניסות וביקורת, צריך הרשאת מנהל). לכל אירוע יש רמה (Information, Warning, Error), תאריך ושעה, מקור (Source), מזהה (Event ID) והודעה. Event ID מזהה סוג אירוע של מקור מסוים, ולכן מתעדים תמיד את שניהם יחד. Filter Current Log מסנן לפי זמן, רמה, מקור ומזהה, וכשיש מאות אירועים זה הכלי שמוצא את הרלוונטי.

כותרת המשנה מבקשת להצליב: בתעודה בודקים תאריך, Subject, Issuer ושרשרת, ומצליבים אותם עם האירועים מאותה שעה ב-Event Viewer כדי לראות מה קרה קודם ומה גרם למה. למשל, אם תעודה פגה ב-08:00, ושגיאות התחברות מתחילות ב-08:00, הקשר ברור. אם התחילו שגיאות Kerberos מיד אחרי אירוע של שירות הזמן על שינוי שעה, החשוד הוא השעון. דוגמאות לאירועים שקשורים לנושאי השיעור, בלוג System ובמקור NETLOGON: 5719 (התחנה לא הצליחה ליצור ערוץ מאובטח עם DC, לרוב בגלל רשת או DNS), ו-3210 (התחנה לא הצליחה להזדהות מול DC, לרוב בגלל חשבון מחשב שלא תואם). בלוג Security, 4625 הוא כניסה שנכשלה ו-4624 כניסה שהצליחה.

ההערה לשקופית: הקפידו לתעד הודעה מלאה, Event ID ושעה. צילום חלון ללא זמן או הקשר מקשה על השוואה. בלחיצה ימנית על האירוע יש Copy ואז Copy Details as Text, שמעתיק את כל השדות כטקסט, וזה עדיף על צילום

מסך.

בשטח. Event Viewer מציג זמן מקומי. כשמשווים לאירועים בשרת, מוודאים שלשני המחשבים אותו אזור זמן, או משווים לפי הזמן ב-UTC שמופיע בתצוגת XML של האירוע (בלשונית Details). ב-PowerShell אפשר לשלוף אירועים עם Get-WinEvent, למשל Get-WinEvent -LogName System -MaxEvents 50.

שאלה	תשובה
יש המון אירועים אדומים ביומן. זו התקלה?	כמעט תמיד לא. בתחנה תקינה יש אירועי Error ו-Warning כל הזמן. מחפשים אירועים מהזמן שבו התחילה התקלה ומהמקור שקשור אליה.

חלופה

אם כלי PowerShell אינו קיים בגרסה המקומית, משתמשים בחלון המקביל ובודקים בו את אותם שדות: Network Connections במקום Get-NetAdapter, Resource Monitor במקום Get-NetTCPConnection, Event Viewer במקום Get-WinEvent.

סימולטור התחקור

שקופית 13 · תרגול אבחון: משתמש מקומי נכנס, משתמש רשתי לא

על המסך: בשמאל חלון PowerShell מדומה, בכותרת Administrator: Windows PowerShell, WS-4487. בימין לוח השלבים: מספר השלב מתוך שישה, שם הבדיקה, למה בודקים אותה, הפקודה, וכפתור ״הרץ את הפקודה״. כותרת המשנה מתארת את הלולאה של כל שלב: מה בודקים ולמה, מריצים, קוראים את הפלט, ובוחרים את הצעד הבא לפי מה שהפלט מראה.

התרחיש. בתחנה WS-4487 משתמש מקומי מצליח להיכנס, ומשתמש רשתי לא. הסימפטום הזה כבר אומר הרבה: התחנה עצמה עובדת, ומשתמש מקומי נבדק מול מאגר החשבונות של התחנה עצמה, בלי DC. כל מה שמשתמש רשתי צריך ומשתמש מקומי לא צריך נמצא בחשד: הרשת עד ה-DC, ה-DNS, השירות, השעה, והאמון בין התחנה לדומיין. ששת השלבים שוללים אותם אחד אחרי השני.

איך הסימולטור עובד. לחיצה על ״הרץ את הפקודה״ מקלידה את הפקודה בחלון ומציגה את הפלט. השורות החשובות מודגשות, ושורות שגיאה מופיעות באדום. מתחת מופיעה קריאת הפלט, ואחריה השאלה ״מה השלב הבא?״ עם שלוש אפשרויות. בחירה שגויה מציגה הסבר קצר ומשאירה את השלב פתוח, כך שאפשר לבחור שוב. בחירה נכונה עוברת לשלב הבא, והפלט של השלבים הקודמים נשאר בחלון. בשלב 6 אין בחירה. מופיע כפתור ״לסיכום״, ובסיכום יש כפתור ״התחל מחדש״, שמנקה את החלון וחוזר לשלב 1. באמצע הריצה אין כפתור איפוס: מי שרוצה להתחיל מחדש באמצע צריך לטעון את המצגת מחדש (F5), והיא נפתחת בשקופית 1. מעבר לשקופית אחרת וחזרה שומר את המצב.

איך להעביר. מההערות לשקופית: הריצו מול הכיתה ותנו לחניכים לבחור את הצעד הבא. הדגישו את סדר הבדיקות: כל שלב שולל סיבה אחת. האפשרויות השגויות הן קיצורי דרך שטכנאים עושים הרבה בשטח, ולכן כדאי לבחור אותן בכוונה מדי פעם ולקרוא את ההסבר. בכל השלבים האפשרות הנכונה היא הכפתור הראשון ברשימה. אל תתנו לכיתה ללמוד את זה: הקריאו את שלוש האפשרויות בסדר אחר, ובקשו הצבעה לפני שאתם לוחצים.

שלב	פקודה	השורה שמראים	הצעד הנכון
1	ipconfig /all	Primary Dns Suffix: unit.local, DNS Servers: 10.20.4.10	לבדוק שהשרת שרשום כ-DNS בכלל עונה
2	ping 10.20.4.10	Sent = 4, Received = 4, Lost = 0 (0% loss)	לשאול את ה-DNS על הדומיין עצמו
3	Resolve-DnsName -Name _ldap._tcp.dc._msdcs.unit.local -Type SRV	רשומת SRV שמחזירה את dc01.unit.local בפורט 389	לבדוק שאפשר באמת לפתוח חיבור ל-LDAP
4	Test-NetConnection dc01.unit.local -Port 389	TcpTestSucceeded : True	לשלול הפרש שעונים לפני שבודקים אמון
5	w32tm /query /status	Phase Offset: 0.0412375s	לבדוק את ערוץ האמון של התחנה מול הדומיין

6	nltest /sc_verify:unit.local	שתי שורות Status = 1787 0x6fb ERROR_NO_TRUST_SAM_ACCOUNT	(אין בחירה, "לסיכום")

שלב 1: הגדרות IP וזהות התחנה. לפני שמאשימים את הדומיין מוודאים שיש כתובת ושער, ובעיקר שה-DNS מצביע על בקר הדומיין ולא על שרת אינטרנט. בפלט: הכתובת 10.20.4.87 עם (Preferred), שער 10.20.4.1, סיומת unit.local, ו-DNS 10.20.4.10, שהוא ה-DC. האפשרויות השגויות: להריץ מיד nltest (מוקדם מדי: אם השרת לא זמין גם בדיקת האמון תיכשל, ולא נדע ממה), ולהחליף כרטיס רשת (יש כתובת, שער ו-DNS, כך שהכרטיס עובד). שאלו את הכיתה מה היה קורה אם ב-DNS Servers הייתה רשומה כתובת של שרת DNS חיצוני: כל השלבים הבאים היו נכשלים כבר בשלב 3.

שלב 2: זמינות בקר הדומיין ברשת. ping מבדיל בין "אין נתיב" לבין "יש נתיב אבל השירות לא עונה". ארבע תשובות, אפס אובדן, זמן של מילישנייה. ה-TTL הוא 128, כצפוי משרת Windows באותה רשת. כאן ההערות מבקשות לעצור: ping תקין עדיין לא אומר שהשירות עונה. האפשרות השגויה "לסמן DNS תקין" היא בדיוק הטעות הזו (Ping בודק ICMP, לא את השירות, ושרת יכול לענות ולא לשרת שאילתות). האפשרות "להריץ tracert לבקר" מיותרת: ה-RTT הוא מילישנייה אחת ואין אובדן, אז אין מה לחפש בדרך.

שלב 3: זיהוי הדומיין ובקר הדומיין. תחנה שמתחברת לדומיין מאתרת את הבקר לפי רשומת SRV, ולכן שואלים בדיוק עליה. השם _ldap._tcp.dc._msdcs.unit.local נראה מאיים, אבל הוא בנוי מחלקים פשוטים: שירות LDAP, בפרוטוקול TCP, של בקרי דומיין (dc), באזור _msdcs שבו Active Directory רושם את הרשומות שלו, בדומיין unit.local. התשובה: dc01.unit.local בפורט 389, ובחלק התחתון של הפלט גם הכתובת של dc01: 10.20.4.10. זו אותה כתובת שרשומה כ-DNS. האפשרויות השגויות: nslookup על אתר אינטרנט (שם כללי לא עוזר, שאלנו על הדומיין וקיבלנו תשובה), ולהסיק שהתקלה ב-DNS (ה-DNS החזיר את התשובה הנכונה).

שלב 4: פתיחת חיבור לשירות עצמו. רשומת DNS רק מצביעה על השרת. חיבור שנפתח לפורט 389 מראה שהשירות מאזין ושאין חוסם בדרך. בפלט TcpTestSucceeded : True, וגם SourceAddress 10.20.4.87, כלומר החיבור יצא מהמתאם הנכון. עד כאן נשללה כל שכבת הרשת. האפשרויות השגויות: לאתחל את התחנה (אתחול יכול להסתיר את הממצא בלי להסביר אותו), ולפתוח קריאה לצוות התשתיות (אין עדיין ממצא להעביר). אם הזמן קצר, ההערות מציעות לעצור כאן ולבקש מהחניכים להציע את שאר הבדיקות. תשובה טובה: שעון ואמון.

שלב 5: הפרש שעונים מול הדומיין. Kerberos פוסל כרטיס כשההפרש עולה על חמש דקות, והתסמין נראה בדיוק כמו בעיית הרשאות. הבדיקה זולה, ולכן שוללים אותה לפני הבדיקה האחרונה. בפלט Source הוא dc01.unit.local, הסנכרון האחרון הצליח באותו בוקר, ו-Phase Offset מראה 0.04 שניות (על מה שהשורה הזו מודדת באמת, ראו "השעון" בעמוד מגבלות הסימולטור). האפשרויות השגויות: לחבר מחדש את התחנה לדומיין עכשיו (אולי יעבוד, אבל לא נדע מה היה שבור ואם זה יחזור), ולאפס את סיסמת המשתמש (התקלה אצל כל המשתמשים הרשתיים, כך שהמשתמש אינו החשוד).

שלב 6: ערוץ האמון בין התחנה לדומיין. גם לתחנה יש בדומיין חשבון, "חשבון מחשב", עם סיסמה שהתחנה והדומיין מכירים ומחליפים מעצמם מדי פעם. בעזרתה התחנה פותחת מול ה-DC ערוץ מאובטח, ודרכו היא מעבירה את בקשות הכניסה של משתמשים רשתיים. אם החשבון חסר בדומיין, או שהסיסמה בתחנה ובדומיין לא תואמות, אין ערוץ, ואף משתמש רשתי לא נכנס. nltest /sc_verify בודק בדיוק את הערוץ הזה. בתשובה תקינה שתי שורות הסטטוס מסתיימות ב-NERR_Success 0x0 0. כאן מופיע בשתיהן קוד 1787 (ERROR_NO_TRUST_SAM_ACCOUNT): בדומיין אין חשבון מחשב תקף לתחנה הזו. וזה גם ההסבר לסימפטום: משתמש מקומי עובד כי הוא לא עובר דרך הדומיין בכלל.

הסיכום שעל המסך: הבדיקות שללו את הכתובת, הנתיב, ה-DNS, השירות והשעה, והצביעו על חשבון המחשב. התיקון שמוצע שם הוא חידוש סיסמת חשבון המחשב או צירוף מחדש לדומיין. שני התיקונים דורשים הרשאות בדומיין ונעשים לפי נוהל היחידה. כשחשבון המחשב נמחק מהדומיין, חידוש סיסמה לא יעזור, כי אין לו חשבון לחדש, וצריך צירוף מחדש או שחזור של החשבון.

טעות נפוצה

הפלט של nltest מסתיים בשורה The command completed successfully, והטכנאי מסיק שה-Domain תקין, אף שהבדיקה עצמה נכשלה. השורה הזו אומרת רק ש-nltest הצליח לבצע את השאילתה. התשובה לשאילתה נמצאת בשורות ה-Status. כאן ההערות מבקשות לעצור בפעם השנייה.

שאלה	תשובה
למה לא לצרף מחדש לדומיין כבר בהתחלה? זה פותר הרבה תקלות.	כי בלי בדיקה לא יודעים מה היה שבור. אם הבעיה הייתה DNS או שעון, הצירוף ייכשל או שהתקלה תחזור. וצירוף מחדש הוא שינוי בדומיין, שלא עושים בלי ממצא.
איך זה קורה, שחשבון מחשב "נעלם" או לא תואם?	סיבות נפוצות: מישהו מחק או השבית את חשבון המחשב בדומיין, תחנה שוחזרה מגיבוי או מתמונה ישנה עם סיסמה ישנה, או שתי תחנות עם אותו שם.
יש דרך לבדוק את זה ב-PowerShell?	Test-ComputerSecureChannel מחזיר True או False. הפרמטר Repair- שלו מתקן, ודורש הרשאות דומיין, ולכן לא משתמשים בו בשיעור.

מה הסימולטור לא מראה, ושרשרת הראיות

אין במצגת שקופית שמפרטת את מגבלות הסימולטור. כדאי להקדיש להן דקה אחרי הסיכום, כי חניכים יוצאים לפעמים מתרגול כזה עם הרושם שתחקור אמיתי הוא שישה צעדים בסדר קבוע עם תשובה אחת נכונה. הסדר והפקודות נכונים, אבל יש בתרגיל כמה פשטות שכדאי להכיר.

- **מסלול אחד.** לכל שלב יש תשובה נכונה אחת. בשטח אפשר להריץ כמה בדיקות במקביל, ולפעמים סדר אחר נכון באותה מידה, למשל להתחיל מ-nltest כשכבר יודעים שהרשת תקינה בתחנה הסמוכה.
- **אין סימפטום ואין יומן.** התרגיל מתחיל משורת הפקודה. בתחנה אמיתית רואים קודם את הודעת השגיאה במסך הכניסה, ובתקלה הזו היא בדרך כלל
  The trust relationship between this workstation and the primary domain failed. ביומן System של התחנה.
  מופיע אירוע NETLOGON 3210. שני אלה מצביעים על ערוץ האמון כבר בהתחלה. הבדיקות בסימולטור הן הדרך לאשר את החשד ולשלול סיבות אחרות, וזה מה שמעבירים הלאה.
- **פלט מקוצר.** כל הפקודות והשדות בסימולטור אמיתיים, אבל חלק מהשורות הושמטו. ב-ipconfig /all חסרים בין השאר Description, Physical Address ו-DHCP Enabled. ב-Resolve-DnsName חסרות העמודות Section, Priority ו-Weight. ב-Test-NetConnection חסר InterfaceAlias.
- **השעון.** הפלט האמיתי של w32tm /query /status ארוך יותר (Precision, Root Delay, ReferenceId, Poll Interval ועוד), והשורה Phase Offset מופיעה בדרך כלל רק כשמוסיפים /verbose. היא גם מתארת את התיקון ששירות הזמן עוד מבצע בשעון, וזו אינה מדידה ישירה מול ה-DC. בשטח, את ההפרש מול ה-DC מודדים ב-
  w32tm /stripchart /computer:dc01.unit.local /samples:3 /dataonly. המסקנה של שלב 5 נשארת נכונה: השעון מסונכרן ל-DC והסנכרון האחרון הצליח.
- **ping ל-DC.** בסימולטור הוא עונה. בשטח בקר דומיין יכול לסנן ICMP, ואז כישלון בשלב 2 לא אומר שאין נתיב. במקרה כזה ממשיכים לשלב 4, שבודק את הפורט ישירות.
- **פורט אחד.** שלב 4 בודק רק LDAP בפורט 389. כניסה של משתמש רשתי משתמשת בעוד פורטים, ובהם DNS (53), Kerberos (88), SMB (445) ו-RPC (135 ועוד פורטים דינמיים). חומת אש שפותחת 389 וחוסמת 88 תעבור את שלב 4 ותכשיל כניסה.
- **הפלט של nltest משתנה לפי הסיבה.** בתקלות אמיתיות השורות יכולות להיראות אחרת: Flags: 0, שורת Trusted DC Name ריקה, וקודים אחרים, למשל ERROR_ACCESS_DENIED 0x5 5 כשהסיסמאות לא תואמות, או ERROR_NO_LOGON_SERVERS 0x51f 1311 כשהתחנה לא מגיעה לאף DC. הכלל לא משתנה: קוראים את שורות ה-Status, ורק NERR_Success 0x0 0 הוא תקין.
- **דומיין עם בקר אחד.** ברשת אמיתית יש בדרך כלל כמה בקרים, ורשומת ה-SRV מחזירה את כולם. תחנה יכולה לעבוד מול בקר אחד ולהיכשל מול אחר. nltest /dsgetdc מראה מול איזה בקר התחנה עובדת כרגע.
- **פרטים שמורים.** בתקלת אמון, משתמש רשתי שכבר נכנס בעבר לתחנה עשוי בחלק מהמקרים להיכנס עם הפרטים השמורים בה. אז נראה שהתקלה ״רק אצל חלק מהמשתמשים״, ומי שבודק את המשתמש במקום את התחנה הולך לכיוון הלא נכון.
- **זמן ותיקון.** בסימולטור כל פקודה מחזירה תשובה מיד. בשטח בדיקה שנכשלת מחכה לפעמים עשרות שניות עד שהיא מוותרת. והתרגיל נגמר באבחון. התיקון עצמו (nltest /sc_reset, Test-ComputerSecureChannel -Repair, או צירוף מחדש, ואחריהם אתחול) דורש הרשאות דומיין ונעשה לפי נוהל היחידה.
- **פרטים קוסמטיים.** חלון PowerShell שנפתח כמנהל מתחיל בדרך כלל בתיקייה C:\Windows\system32.
  בסימולטור שורת הפקודה מציגה את תיקיית המשתמש. תבנית התאריך בפלט של w32tm תלויה בהגדרות האזור של התחנה.

מה מעבירים הלאה

מה שהסימולטור כן מלמד נכון: כל שלב שולל סיבה אחת, ממצא תקין לא מוכיח את השלב הבא, ושורת הסיום של פקודה אינה התשובה שלה. למי מעבירים ממצא כזה ובאיזו צורה נקבע לפי נוהל היחידה. כדאי לצרף את הפקודות שהורצו ואת שורות הסטטוס כטקסט, עם השעה.

שקופית 14 · שרשרת הראיות מהמתאם עד הזהות

על המסך: חמש תחנות ברצף: Get-NetAdapter (יש Link), ipconfig / arp (זהות ושכן), route / tracert (יש דרך), DNS / Port (יש שם ושירות), Domain / Cert (יש אמון). כותרת המשנה: עוצרים ובודקים לעומק רק בשלב שבו התוצאה אינה תואמת לתכנון או לייחוס.

רקע. זו המפה של שקופית 2, מקובצת לחמש חוליות. כל חוליה נשענת על זו שלפניה, וכל כלי מוסיף ראיה אחת. אם הממצא תואם לתכנון ממשיכים, ואם לא, עוצרים ובודקים לעומק. הסימולטור עבר בדיוק בדרך הזו. חמש בדיקות תאמו לצפוי, והשישית חרגה.

לאורך כל השיעור משווים לתכנון של הרשת או לתחנה תקינה שעומדת ליד. בלי משהו להשוות אליו, אי אפשר לדעת אם 10.20.30.15 או 00-15-5d-01-02-03 נכונים.

איך להעביר. ההערה לשקופית: אין צורך בתרגול מסכם. סיימו במפה שבשקף ועברו בקצרה על מה שכל כלי בדק. אפשר לעבור על החוליות מימין לשמאל ולשאול על כל אחת: איזו פקודה, איזה שדה, ומה עוד לא הוכח.

שאלת סיום

איזו תוצאה של Test-NetConnection מוכיחה פתיחת TCP, ומה היא אינה מוכיחה לגבי היישום? (TcpTestSucceeded : True. היא לא מוכיחה שהיישום מחזיר תוכן תקין, שהתעודה תקינה או שהמשתמש מורשה.)

מפת המצגת, ערכי בדיקה ומילון

מפת המצגת

מס׳	כותרת השקופית	מס׳	כותרת השקופית
1	כלי Windows: מה בודקים בכל פקודה (שער)	8	Resolve-DnsName או nslookup בודקים את ה-DNS בנפרד מהשירות
2	איזה כלי עונה על איזו שאלה	9	Test-NetConnection בודק אם ניתן לפתוח חיבור TCP לפורט מסוים
3	Get-NetAdapter מראה אם Windows מזהה מתאם, Link וקצב קישור	10	Get-NetTCPConnection או netstat מראים מי מאזין ומי מחובר
4	ipconfig /all מציג את הזהות המקומית של התחנה ברשת	11	בדיקות Kerberos: זמן, איתור DC וכרטיסים
5	Ping בודק תשובת ICMP ומודד RTT	12	Certificate Viewer ו-Event Viewer
6	arp -a מראה איזה MAC משויך ל-IP המקומי או ל-Gateway	13	תרגול אבחון: משתמש מקומי נכנס, משתמש רשתי לא
7	route print ו-tracert בודקים את המסלול	14	שרשרת הראיות מהמתאם עד הזהות

ערכי בדיקה להדגמות

הערכים שעל המסך קבועים. אם רוצים לוודא לפני השיעור שהכול עובד, אלה התוצאות שצריכות להופיע:

שקופית	מה בודקים	מה צריך להופיע
7 · route print	איזה שער ייבחר ליעד 10.20.40.25?	10.20.30.2 (השורה הספציפית). כל יעד אחר: 10.20.30.1
13 · שלב 1	ipconfig /all	10.20.4.87, שער 10.20.4.1, DNS 10.20.4.10
13 · שלב 2	ping 10.20.4.10	4 תשובות, 0% loss, TTL=128
13 · שלב 3	רשומת SRV של בקר הדומיין	dc01.unit.local, פורט 389, TTL 600, 10.20.4.10
13 · שלב 4	פורט 389	TcpTestSucceeded : True
13 · שלב 5	שעון	Source: dc01.unit.local, Phase Offset: 0.0412375s
13 · שלב 6	ערוץ אמון	ERROR_NO_TRUST_SAM_ACCOUNT 0x6fb 1787, ובסוף The command completed successfully
13 · כל שלב	האפשרות הנכונה	הכפתור הראשון ברשימה. כל בחירה שגויה מציגה הסבר ומשאירה את השלב פתוח

מילון קצר

מונח	הסבר	מונח	הסבר
Status Up	המתאם רואה קישור פיזי.	**RTT**	זמן הלוך ושוב של חבילה.
TTL	מונה קפיצות בחבילת IP. ב-DNS: כמה זמן מותר לשמור תשובה.	**ARP**	מציאת MAC לפי IP ברשת המקומית.
Route	כלל בחירת דרך: לאיזה שער שולחים יעד מסוים.	**Gateway**	הנתב שאליו נשלחת תעבורה לרשתות אחרות.
Cache	מידע שמור זמנית, למשל תשובות DNS.	**SRV**	רשומת DNS לאיתור שירות: שרת ופורט.
Listener	שירות שמאזין לפורט (LISTENING).	**Established**	חיבור TCP שנוצר.
PID	מספר התהליך שמחזיק פורט או חיבור.	**DC**	בקר דומיין: השרת שמאמת משתמשים ומחשבים.
TGT	כרטיס Kerberos ראשי, שממנו מבקשים כרטיסים לשירותים.	**ערוץ אמון**	הערוץ המאובטח בין התחנה ל-DC, לפי חשבון המחשב.
Event ID	מזהה סוג אירוע של מקור מסוים.	**ייחוס**	תחנה או ערך תקין שמשווים אליו.